

RELATÓRIO AULA 007 MUNIF

A aula começou com a exposição de seus objetivos principais, mostrando que interpretar expressões de maneira correta e manter um controle cuidadoso sobre o fluxo de execução evita falhas graves em sistemas reais. Para isso, foram citados casos conhecidos do mercado, como o "goto fail" da Apple, o erro do ano bissexto no Zune e a brecha causada pela coerção de tipos no PHP.

Em seguida, os alunos responderam a um quiz de abertura, cujo propósito era medir o conhecimento prévio da turma e coletar palpites sobre situações-limite envolvendo precedência de operadores, conversão implícita de tipos, expressões relacionais encadeadas e regras de avaliação booleana em linguagens como Python, Java, JavaScript e C.

Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela indentação ou pela precedência
- evitar conversões implícitas perigosas (== do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle

2014 · "goto fail" (Apple)

Uma linha `goto fail`; duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um laço `while` nunca terminava, e todos os aparelhos travaram.

PHP · comparação com ==

`"0e4..." == "0e8..."` é verdadeiro: logins que comparavam hashes com == aceitavam senhas erradas.

Com isso, foi possível compreender o rumo que a aula tomaria e os conceitos ligados a ela, já que o conteúdo busca resolver erros frequentes na rotina de programação, que costumam surgir por falta de hábito ou de conhecimento sobre a linguagem utilizada.

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

Palpites: imaginei que, seguindo a ordem numérica dos trechos, as saídas seriam: “-4, 512” no primeiro; 3”3”9 no código Java; no JavaScript, a realização de uma soma, resultando em 3 e 7; nada seria impresso pela linguagem C; o Python imprimiria x; e o último trecho em JavaScript imprimiria false, true, true.



```
main.py × +  
1 print(-2 ** 2, 2 ** 3 ** 2)  
2  
-4 512
```

No Python, o operador de potência (**) é prioritário em relação ao sinal de menos unário; por isso, `-2 ** 2` equivale a `-(2 ** 2)`, ou seja, -4. Além disso, a potenciação é associativa à direita, de modo que `2 ** 3 ** 2` é resolvido como `2 ** (3 ** 2) = 2 ** 9 = 512`.



```
Main.java × +  
1 import java.util.*;  
2  
3 public class Main {  
4     public static void main(String[] args) {  
5         System.out.println(1 + 2 + "3" + 4 + 5);  
6     }  
7 }  
8  
3345
```

Em Java, as operações são resolvidas da esquerda para a direita. Primeiro, `1 + 2` é calculado como uma soma aritmética e resulta em 3. Depois, ao juntar esse valor com a string "3", acontece uma concatenação, que produz "33". Os números seguintes, 4 e 5, também passam a ser anexados como texto, formando "3345".



```
index.js × +  
console.log("5" - 2, "5" + 2);  
3 52
```

No JavaScript, a operação de subtração força a transformação da string "5" no número 5, de modo que `5 - 2` resulta em 3. Já o operador de adição faz concatenação sempre que um

dos operandos for uma string, e por isso "5" + 2 produz "52".

```
C Main.c x +
1 #include <stdio.h>
2 int main()
3 {
4     printf("%d\n", 3 > 2 > 1);
5 }
```

Na linguagem C, a comparação encadeada $3 > 2 > 1$ é processada da esquerda para a direita. A primeira etapa, $3 > 2$, é verdadeira e vale 1. Na sequência, compara-se $1 > 1$, o que é falso e vale 0. Assim, o valor exibido é 0.

OneCompiler Upgrade AI Python Run

```
main.py x +
1 print(x or "padrão", 3 > 2 > 1)
2
```

Traceback (most recent call last):
File "/box/main.py", line 1, in <module>
 print(x or "padrão", 3 > 2 > 1)
 ^
NameError: name 'x' is not defined

Ao executar a expressão em Python sem que a variável 'x' tenha sido definida antes, o interpretador gera um NameError ('x' is not defined) e encerra a execução antes mesmo de chegar à avaliação lógica do operador 'or'.

```
JS index.js x +
1 console.log(0 == "", "" == "0", 0 == "0");
2
```

Console I/O AI Agent

true false true

No JavaScript, o operador de igualdade ampla ($==$) converte os tipos de forma implícita: $0 == ""$ é verdadeiro, já que os dois lados são convertidos para 0; $"" == "0"$ é falso, pois se trata de duas strings com conteúdos distintos; e $0 == "0"$ é verdadeiro, porque a string é transformada em número. Esse comportamento evidencia que a igualdade ampla não é transitiva.

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

$a + b * c$ com $a=3, b=4, c=5$

da esquerda para a direita

$(3 + 4) * 5 = 35$

da direita para a esquerda

$3 + (4 * 5) = 23$

com precedência (* antes de +)

$3 + 4 * 5 = 23$

APL: sempre da direita para a esquerda (p. 304)

$3 \times 4 + 5 = 27$

Panorama das principais questões de projeto envolvidas na avaliação de expressões aritméticas, comparando a maneira como diferentes linguagens lidam com precedência, associatividade e notações alternativas, como as de Lisp e APL.

O que dizem as regras de precedência?

As regras de precedência definem em que sequência os diversos operadores de uma expressão são avaliados.

Ordem padrão: na maior parte das linguagens, os operadores com precedência mais alta (como multiplicação e divisão) são resolvidos antes dos de precedência mais baixa (como adição e subtração), o que leva a $3 + (4 * 5) = 23$.

Abordagens alternativas:

Lisp: abre mão da precedência convencional ao adotar a notação prefixa, com parênteses explícitos que determinam a ordem exata das operações $((+ a (* b c)))$.

APL: também não segue a precedência convencional; as expressões são sempre resolvidas da direita para a esquerda $(3 \times 4 + 5 = 27)$.

O que dizem as regras de associatividade?

A associatividade estabelece como operadores vizinhos, de mesmo nível de precedência, são agrupados e avaliados.

Da esquerda para a direita: as operações são agrupadas começando pelo lado esquerdo (ex.: $(3 + 4) * 5 = 35$).

Da direita para a esquerda: as operações são agrupadas começando pelo lado direito (ex.: $3 + (4 * 5) = 23$).

Em qual ordem os operandos são avaliados?

Diz respeito à sequência em que os operandos de um operador, ou os argumentos de uma função, são calculados.

Embora a maior parte das linguagens avalie os operandos da esquerda para a direita, algumas deixam o compilador livre para modificar essa ordem com o objetivo de otimizar o código.

Existem restrições quanto a efeitos colaterais?

Há efeito colateral quando avaliar uma expressão altera o valor de alguma variável ou o estado do programa, como ocorre com funções que modificam estado ou com operadores de incremento, como $x++$, usados dentro de uma expressão.

Certas linguagens aplicam limites severos sobre onde ou quando os efeitos colaterais podem acontecer, a fim de impedir que o resultado da expressão varie conforme a ordem em que os operandos são avaliados.

É permitido ao usuário sobrecarregar operadores?

Trata-se de saber se a linguagem deixa o programador atribuir novos significados a operadores já existentes (como +, - e *) para que funcionem com tipos criados pelo próprio usuário (como matrizes, números complexos ou objetos personalizados). Esse recurso existe em linguagens como C++ e Python, mas não em todas.

Quais misturas de tipos são aceitas?

Aborda a forma como a linguagem trata operações que envolvem tipos de dados distintos, como somar um inteiro a um número de ponto flutuante.

A linguagem pode adotar a coerção, ou seja, uma conversão implícita feita automaticamente pelo compilador ou interpretador, ou então exigir tipagem estrita, em que qualquer mistura de tipos demanda uma conversão explícita (cast) escrita pelo programador.

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
-2 ** 2	-4	erro de sintaxe	(não há **)
2 ** 3 ** 2	512	512	—
1 + 2 + "3"	TypeError	"33"	"33"
-7 / 2 (inteiros)	-4 (com //)	-3.5	-3
-7 % 2	1	-1	-1

Associatividade (p. 303)

Esquerda: * / % + -

Direita: ** (Python, JS, Ruby, Fortran), ++, --, menos unário

Visual Basic: ^ à esquerda!

Excel

= -2^2 dá 4: lá o menos unário vence o ^.

Na dúvida, use **parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

É possível notar que as associações variam conforme a linguagem. Por exemplo, quando dois operadores aparecem lado a lado e têm a mesma precedência, cada linguagem decide à sua maneira como resolvê-los, e isso pode ser observado no slide.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Análise do efeito colateral: quando uma função modifica variáveis globais ou externas enquanto a expressão é avaliada (como a fun1 alterando 'a'), o resultado final passa a depender da ordem em que a linguagem avalia os operandos. O Java assegura a avaliação da esquerda para a direita, o que gera 8; já linguagens como C e Go não fixam uma ordem rígida para ler 'a' antes ou depois da chamada da função, e o resultado é 20.

Operadores sobrecarregados

- O mesmo símbolo com vários significados: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- Definida pelo usuário (p. 404): C++, C#, Python, Rust, Kotlin. Java não permite

Java

```
System.out.println('a' + 1);      // 98
System.out.println("" + 'a' + 1); // a1
x.add(y).multiply(tres);          // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):      # v1 + v2
        return Vetor(self.x + outro.x,
                       self.y + outro.y)

    def __mul__(self, k):          # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)     # Vetor(4, 6)
print(v * 3)                     # Vetor(12, 18)
```

A análise mostra que um mesmo operador pode executar ações diferentes sem alterar sua aparência, graças à sobrecarga, recurso que também existe em Java e permite tanto concatenar strings quanto realizar a soma propriamente dita.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;           // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com a digitado por engano no lugar de c, o Java converte e compila.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + "a" + "a" // "baNaNa"

0 == ""           // true
"" == "0"         // false
0 == "0"          // true (não é transitivo!)
0 === "0"         // false (=== não converte)
```

PHP · "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01"); // true

md5("240610708") // "0e462097431906...."
md5("QNKCDZO") // "0e830400451993...."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com `===` e `!==`, que não fazem coerção.

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));    // 1

if (-1) printf("verdadeiro\n");    // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)    # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0    0.0    ""    []    {}    None
# contam como VERDADEIRO:
# "0"    [0]    " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano : <code>if (n)</code> com <code>n</code> inteiro nem compila

Exemplo de sobrecarga de operadores: um mesmo símbolo (como `+`) pode se comportar de várias formas, conforme o tipo dos operandos. C++, Python e Rust permitem que o usuário sobrecarregue operadores em tipos personalizados, ao passo que o Java limita a sobrecarga aos recursos nativos da própria linguagem.

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;    // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0    // false
s != null & s.length() > 0    // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade    // undefined, sem erro

const qtd = 0;
qtd || 10    // 10 (o 0 se perdeu!)
qtd ?? 10    // 0 (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. Em Python, `0` or "padrão" devolve "padrão": and e or devolvem um dos operandos.

Exemplos de conversão de tipos (alargamento e estreitamento) e de coerção implícita. Enquanto C e Java aplicam coerções automáticas em determinadas operações, linguagens modernas e estritamente tipadas, como Go e Rust, exigem conversões explícitas (cast) para evitar erros silenciosos.

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... }          // compila! atribui e testa y
```

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: <code>if (x = y)</code> compila
Java, C#	sim, mas o <code>if</code> exige booleano
Python	não, exceto o operador morsa <code>:=</code>
Go	não: nem <code>x++</code> é expressão
Rust	vale <code>()</code> : <code>if x = 5</code> não compila

Linguagens funcionais (p. 321)

`val` (ML) e `let` (F#, Rust) dão **nome** a um valor, que não muda.

Demonstração dos riscos da coerção implícita em JavaScript e PHP (type juggling). Nesses casos, comparações mais elaboradas com `==` podem produzir resultados pouco intuitivos e abrir brechas de segurança em validações. Como regra prática, convém usar sempre os operadores de igualdade estrita (`===` e `!==`).

Atribuição de modo misto

Java · só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;          // erro: estreitamento
int i = (int) 3.7;       // 3 (cast explícito)
long l = i;              // ok

i += 3.7;                 // COMPILA: cast escondido
System.out.println(i);    // 6
```

C · coerção em qualquer direção

```
int i = 3.7;             // 3, em silêncio
char c = 300;            // 44 (só um aviso)
unsigned int u = -1;      // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100`; com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para `double`.

Análise das expressões relacionais e booleanas: contrasta o tratamento de valores booleanos como inteiros (caso do C) com os tipos booleanos estritos (Java, Go e Rust), além de mostrar que o Python aceita comparações encadeadas (`a > b > c`) e considera falsos objetos e coleções vazios.

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;  
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...  
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)  
    x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade:** quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Explicação sobre a avaliação em curto-circuito: em expressões lógicas condicionais (&&, ||), o segundo operando só é avaliado quando o primeiro não basta para definir o resultado final. Empregar de forma inadequada operadores sem curto-circuito (como o & em Java) pode provocar exceções, como a NullPointerException.

Seleção de dois caminhos: o "else pendente"

C · a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;  
if (sum == 0)  
    if (count == 0)  
        result = 0;  
else  
    result = 1;          // de qual if é este else?  
printf("%d\n", result); // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if mais próximo que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };  
y = x if x > 0 else 2 * x
```

// Rust: if é uma expressão (como em F#, p. 334)
Python: expressão condicional (p. 306)

Exemplo prático de avaliação em curto-circuito em Java e JavaScript: em expressões com operadores lógicos (como && ou ||), a avaliação é interrompida assim que o resultado se torna conhecido, o que evita erros como a NullPointerException. Também são apresentados o operador de encadeamento opcional (?) e o de coalescência nula (??), usados para lidar com segurança com valores null e undefined.

Estudo das sentenças de atribuição: compara linguagens em que a atribuição é uma expressão que devolve valor (o que permite atribuições encadeadas ou atribuições dentro de condições, como em C/C++) com linguagens funcionais ou de tipagem mais rígida, nas quais a atribuição é apenas uma instrução.

Laços controlados por contador

C · o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}  
  
for (; i <= 100; ) // expressões opcionais  
    soma += i++;
```

Python · o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
                    # 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Quanto aos laços controlados por contador, nota-se que eles se comportam de maneira diferente conforme a linguagem e o que se deseja fazer. No primeiro exemplo, o contador é incrementado depois da entrada no laço e também por uma variável alterada dentro dele; já o segundo exemplo mostra que a iteração decorre de uma definição prévia de range, que igualmente conduz a repetição de acordo com a função do Python.

Laços lógicos, saídas e iteradores

Java · break rotulado (p. 348)

```
externo:  
for (int i = 0; i < m.length; i++)  
    for (int j = 0; j < m[i].length; j++)  
        if (m[i][j] < 0) {  
            System.out.println(i + ", " + j);  
            break externo; // sai dos DOIS laços  
        }
```

JavaScript · for...in × for...of

```
const notas = [10, 20, 30];  
let total = 0;  
for (const i in notas) total += i;  
console.log(total); // "0012": chaves!  
  
total = 0;  
for (const v of notas) total += v;  
console.log(total); // 60: valores
```

Zune (2008) · o laço lógico sem progresso

```
while (dias > 365) {  
    if (bissexto(ano)) {  
        if (dias > 366) { dias -= 366; ano++; }  
        // dias == 366: nada muda, laço infinito!  
    } else { dias -= 365; ano++; }  
}
```

Pós-teste (do-while) executa ao menos uma vez, e isso é fácil de esquecer (p. 347).

Análise de laços lógicos e iteradores: ressalta a necessidade de um avanço explícito em laços condicionais para evitar repetições infinitas (como a falha do Zune), o uso de rótulos em laços Java (break rotulado) para sair de estruturas aninhadas e a diferença, em JavaScript, entre 'for...in' (que percorre chaves) e 'for...of' (que percorre valores).

Desvio incondicional: o goto

"goto fail" (Apple, 2014) • versão simplificada

```
int valida_certificado(void) {  
    int err;  
    if ((err = verifica_hash()) != 0)  
        goto fail;  
    goto fail;           // duplicada: desvia SEMPRE  
    if ((err = verifica_assinatura()) != 0)  
        goto fail;       // nunca é alcançada  
fail:  
    return err;           // 0 = "certificado válido"  
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java não tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Análise do perigo dos desvios incondicionais (comando goto), exemplificada pelo conhecido erro de segurança "goto fail" da Apple, em 2014, no qual uma instrução repetida fazia o programa ignorar a verificação da assinatura digital. O slide coloca lado a lado a visão histórica de Dijkstra e o projeto de linguagens modernas, que evitam o goto ao exigir blocos estruturados com chaves ou ao oferecer apenas saltos rotulados e controlados (break, continue, return).

Comandos protegidos de Dijkstra

```
if x >= y -> max := x  
[] y >= x -> max := y  
fi
```

Exemplo do livro (p. 354): se $x == y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no if) ou fim do laço (no do ... od)
- Base do CSP de Hoare (1978) e do select de Go

Go • select, herdeiro dos comandos protegidos

```
a <- "a"           // os dois canais  
b <- "b"           // estão prontos  
  
select {           // Go sorteia uma guarda  
case x := <-a:  
    contagem[x]++  
case x := <-b:  
    contagem[x]++  
}  
  
// 1000 repetições:  
// map[a:505 b:495] (muda a cada execução)
```

Exame mais detalhado dos riscos dos desvios incondicionais (comando goto), representados pelo célebre erro de segurança "goto fail" da Apple, de 2014, em que uma linha duplicada anulava a checagem da assinatura digital. O slide confronta o ponto de vista histórico de Dijkstra ("Go To Statement Considered Harmful") com a filosofia de linguagens modernas, que evitam ou limitam o goto, seja exigindo blocos estruturados com chaves, seja disponibilizando somente saltos rotulados e controlados (break, continue, return).

Discussão sobre as regras de atribuição em modo misto e sobre a coerção entre tipos numéricos de precisões diferentes. Destaca-se a clássica armadilha da divisão inteira realizada antes da atribuição a um tipo de ponto flutuante, que faz a precisão se perder antes mesmo de o tipo ser alargado.

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):
    d = d or 10    # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Análise do teorema de Böhm e Jacopini a respeito das estruturas de controle fundamentais (sequência, seleção e iteração), indicando que construções adicionais (for, switch, do-while) existem para tornar a escrita mais prática e o código mais legível.

Exercício: encontre o bug (2/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Análise do problema do "else pendente" em estruturas de seleção aninhadas. Em linguagens como C, C++ e Java, o 'else' é vinculado automaticamente ao 'if' mais próximo que ainda não tenha um 'else', não importando como o código está endentado. Linguagens modernas, como Go, Rust e Perl, eliminam essa ambiguidade ao tornar as chaves obrigatórias.

Exercício: encontre o bug (3/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Apresentação dos comandos protegidos de Dijkstra, um modelo não determinístico em que todas as guardas são avaliadas ao mesmo tempo. Quando mais de uma é verdadeira, o caminho é escolhido de forma aleatória, o que serve de base conceitual para construções como o comando 'select' do Go.

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>==</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	<code>if</code> só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	<code>switch</code> sem <code>break</code> ; <code>else</code> pendente	<code>-></code> , <code>match</code> exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, <code>break</code> rotulado
Desvio	<code>goto</code> sem restrição	<code>break</code> , <code>continue</code> , <code>return</code> : gotos restritos

Exercício Encontre o Bug (1/3)

1. JavaScript (uso de `for...in` em arrays)

(a) Saída: "Total: 0012"

(b) Regra: em JavaScript, o laço 'for...in' percorre as chaves ou índices do objeto/array, tratados como strings ("0", "1", "2"), e não os valores numéricos. Ao somar esses índices a 'total' (que começa em 0), ocorre coerção de tipo e as strings são concatenadas.

(c) Correção: trocar por 'for...of', que percorre os valores do array: 'for (const p of precos) { total += p; }'.

2. Python (argumento padrão recebendo um valor falso)

(a) Saída: 180.0

(b) Regra: no Python, o número 0 é considerado Falso em contexto booleano. Quando se passa `d=0`, a expressão `'d = d or 10'` avalia `'0 or 10'` e resulta em 10, aplicando 10% de desconto em vez de 0%.

(c) Correção: verificar de forma explícita se o valor é None: `'if d is None: d = 10'`.

Exercício Encontre o Bug (2/3)

3. Java (efeito fall-through no switch)

(a) Saída: "inválido"

(b) Regra: o 'switch' tradicional do Java tem comportamento de 'fall-through'. Como nenhum 'case' termina com 'break', a execução segue adiante e realiza todas as atribuições seguintes até chegar ao bloco 'default'.

(c) Correção: incluir 'break;' no fim de cada 'case' ou recorrer à sintaxe moderna do switch (Java 14+): `'case 2 -> nome = "segunda";'`.

4. C (ponto e vírgula logo após a condição do if)

(a) Saída:

saldo insuficiente

fim

(b) Regra: o ponto e vírgula colocado logo depois da condição do 'if' (`'if (saque > saldo);'`) fecha o corpo da estrutura como uma instrução nula. Com isso, o printf seguinte é executado sempre, sem depender da condição.

(c) Correção: eliminar o ponto e vírgula depois do 'if': `'if (saque > saldo) printf(...).'`

Exercício Encontre o Bug (3/3)

5. Go (ocultação de variável, ou shadowing)

(a) Saída:

dentro: 20

fora: 10

(b) Regra: o operador de declaração curta em `'x := x * 2'`, dentro do bloco 'if', cria uma nova variável local 'x' que oculta (shadowing) o 'x' do escopo externo. Por isso, a modificação não atinge a variável de fora.

(c) Correção: empregar a atribuição simples no lugar da declaração: `'x = x * 2'`.

6. Java (divisão inteira realizada antes da atribuição a um double)

(a) Saída: 0.0%

(b) Regra: como 'acertos' e 'total' são inteiros, a operação `'acertos / total'` é uma divisão inteira (`7 / 10 = 0`), efetuada antes da multiplicação por 100 e da conversão para double na atribuição.

(c) Correção: assegurar que pelo menos um dos operandos seja de ponto flutuante: `'double taxa = (double) acertos / total * 100;'` ou `'double taxa = acertos * 100.0 / total;'`.

Síntese do Conteúdo

Resumo das principais armadilhas abordadas nas linguagens de programação e dos mecanismos modernos criados para atenuá-las, cobrindo precedência, efeitos colaterais, coerção, avaliação booleana, comandos de seleção e controle de fluxo.